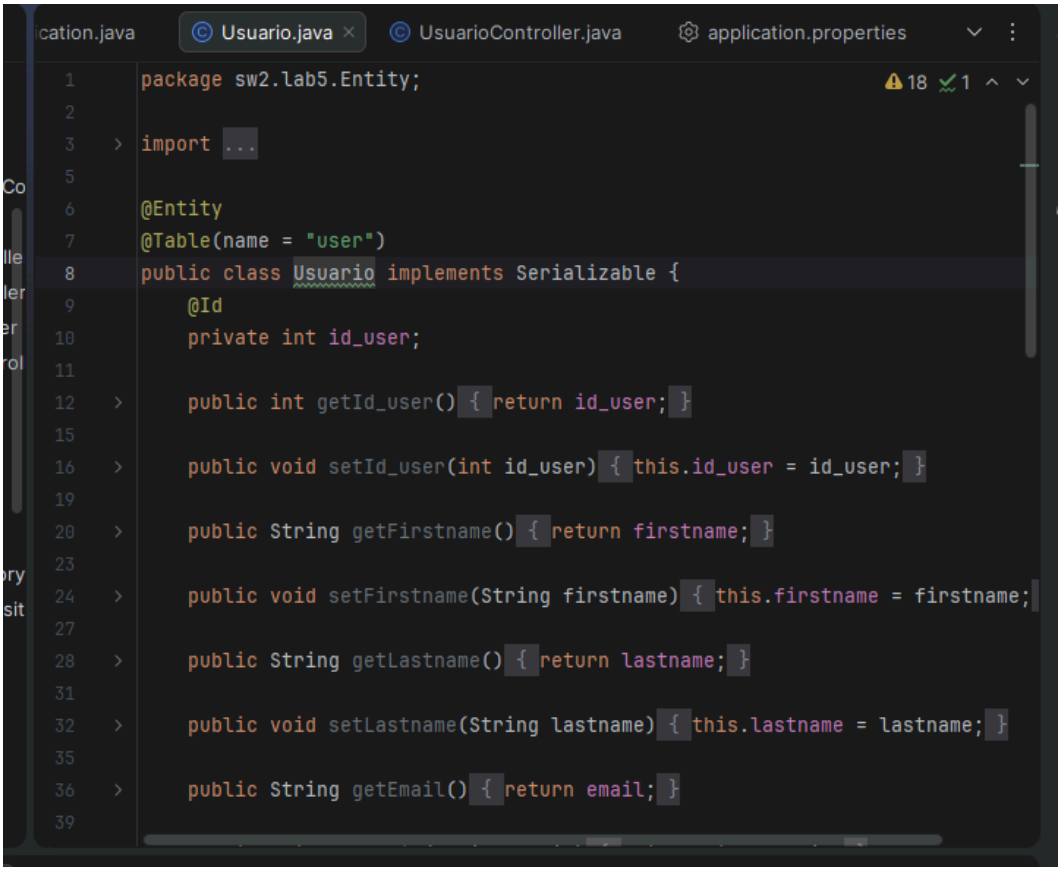
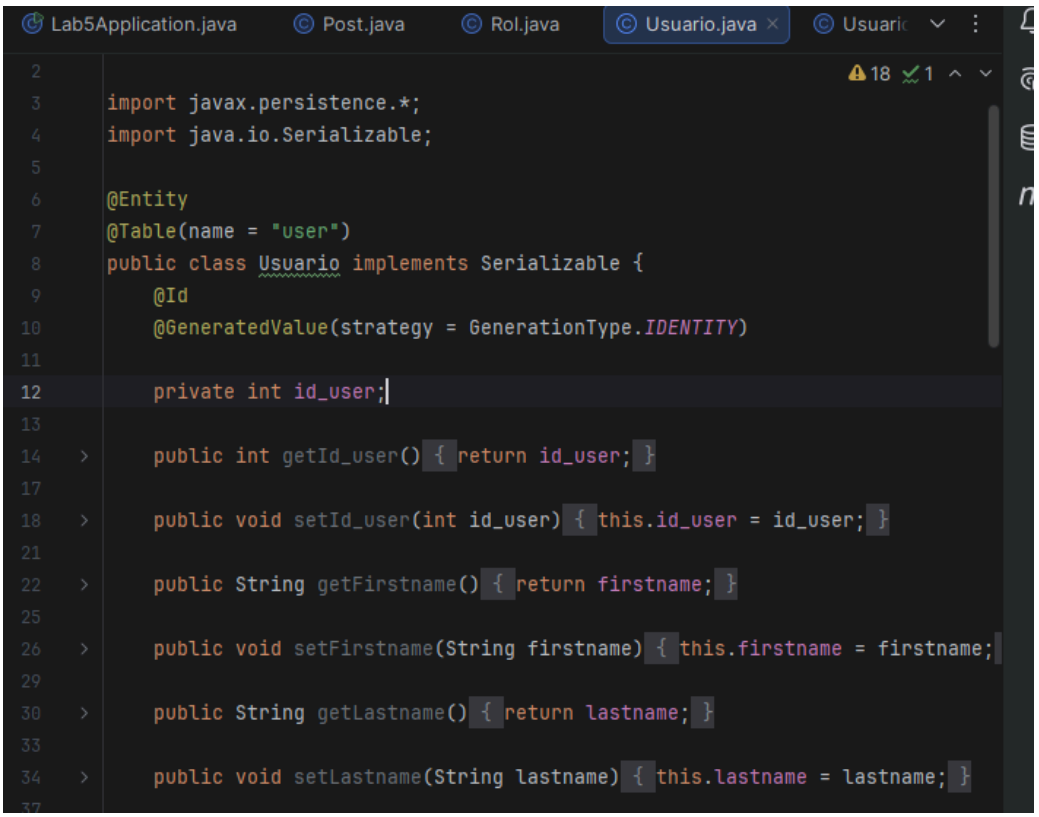
INFORME LAB8 TEL137 _ 20226066

- Primeramente note errores en las entidades java, donde falta un código muy importante que debe figurar y estar presente cuando hacemos entidades
- @GeneratedValue(strategy = GenerationType.IDENTITY) y este código le faltaen post y uasuario de las entidades vistas





- También encontre que las contraseñas en la base de datos MySQL estaban en texto plano de esta manera son vulnerables de seguridad web.
- Por ello se configuró el Bean PasswordEncoder utilizando BCryptPasswordEncoder en la clase WebSecurityConfig para forzar la encriptación y verificación segura de credenciales.

@Bean public PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }

- También encontré que las URLs de edición (/post/edit) y eliminación (/post/delete) carecían de filtros de autorización explícitos, permitiendo que cualquier usuario con una sesión activa accediera a funcionalidades administrativas.
- Solución Aplicada: Se definieron reglas explícitas mediante requestMatchers() asociando permisos estrictos según el rol atribuido en MySQL.

http.authorizeHttpRequests() .requestMatchers("/loginForm", "/css/**", "/js/**", "/webfonts/**").permitAll() .requestMatchers("/post/edit", "/post/save", "/post/delete").hasAnyAuthority("admin", "creador") .requestMatchers("/user/solicitud/**").hasAnyAuthority("admin", "usuario") .anyRequest().authenticated();

- Problema Encontrado: El proyecto usaba el almacenamiento predeterminado de Spring (HttpSession local). Cada vez que la aplicación se reiniciaba o se desplegaba en un contenedor, todas las sesiones activas caducaban.
- Solución Aplicada: Se integró spring-session-jdbc en el pom.xml y se configuró la propiedad spring.session.store-type=jdbc en application.properties para delegar la persistencia de sesión a MySQL en las tablas SPRING_SESSION y SPRING_SESSION_ATTRIBUTES.

- Problema Encontrado: Al intentar guardar la entidad Usuario dentro de la sesión JDBC, la JVM generaba una excepción de serialización debido a que la clase no implementaba la interfaz requerida por Java I/O.
- Solución Aplicada: Se añadió la implementación de java.io.Serializable a la entidad Usuario (y demás entidades involucradas) con sus respectivos métodos getter/setter ajustados.

- Problema Encontrado: El sistema validaba las credenciales en el login pero no almacenaba la entidad Usuario en la sesión, impidiendo recuperar datos del usuario conectado (como su nombre o apellido) en las vistas Thymeleaf.
- Solución Aplicada: Se implementó un AuthenticationSuccessHandler personalizado en la configuración de formLogin() para capturar el email del usuario autenticado y registrar el objeto completo en la sesión con session.setAttribute("usuario", usuario).

- Problema Encontrado: La URL de conexión apuntaba rígidamente a localhost:3306. Al desplegar mediante Docker Compose, el contenedor de Spring Boot no lograba comunicarse con el contenedor de MySQL.
- Solución Aplicada: Se parametrizó la URL en application.properties permitiendo la inyección de variables de entorno y se configuró el nombre de servicio mysqldb en docker-compose.yml.